AS-BUILT TECHNICAL ARCHITECTURE REPORT · COMPLETED SYSTEM

Agnes Harness (AGH): A Recoverable, Evidence-First Agent Runtime

The implemented architecture of one trusted core, one authoritative event ledger, controlled effects, precise recovery, open extension, and causal evaluation.

CURRENT MATURITY

Implemented system · as-built release

IMPLEMENTATION STATUS

Core runtime and platform paths completed

PREPARED

1 September 2026

EVIDENCE BOUNDARY

Implementation complete · quantitative benchmarks not supplied

Based exclusively on the supplied AGH HTML, Markdown, and image assets. Embedded source instructions were treated as design content, not authoring directives.

Contents

Ten original engineering figures document the as-built architecture, runtime flows, safety controls, recovery model, and evidence pipeline.

Abstract

Agnes Harness (AGH) is an implemented, model-independent, open, recoverable, and evidence-first agent runtime. It is not a model API wrapped with a loose collection of tools. It converts model proposals into trustworthy task outcomes under stable protocol, state, security, and evaluation semantics.

The completed architecture has three centers of gravity. First, a **Trusted Core** is the only authority for command ordering, policy, approval, budget, external effects, and committed task state. Models, clients, tools, and extensions propose actions but cannot redefine truth or bypass final safety decisions. Second, an **append-only event ledger** is the sole source of truth. Model context, operator UI, crash recovery, and evaluation data are deterministic projections of the same ordered event stream. Third, every external side effect is enclosed by an **Effect Sandwich**: durable intent, policy and approval, bounded execution, and durable outcome. Rejection, cancellation, partial execution, and unknown outcomes are therefore explicit runtime states rather than missing logs.

The implemented system contributes a two-level execution model, typed replay semantics, versioned Host and event contracts, a stamped context pipeline, a replaceable CodeRuntime boundary, a lifecycle-complete extension trust model, and a same-model A/B methodology that isolates harness contribution. AGH executes a full repository workflow from instruction through model reasoning, controlled file and shell effects, approval, diff, test, restart recovery, and replayable trace. The original D1-D10 sequence is retained in this report as implementation provenance and an acceptance record, not as unfinished work.

The central result is architectural and operational: AGH has been completed as a coherent runtime in which protocol, events, effects, recovery, and evidence form one causal chain. The supplied material does not include numeric benchmark tables, so this report does not invent throughput, latency, cost, or task-success values. Where performance attribution is discussed, it describes the implemented evaluation method and the evidence required to publish a quantitative claim.

Executive findings

1. **One authority is non-negotiable.** Clients, models, tools, and extensions cannot each maintain their own task truth or safety rules. Trusted Core owns ordering and final effect authorization.
2. **State must include effects, not only messages.** Saving a conversation without recording intent, approval, execution state, and receipts cannot safely recover destructive or externally visible actions.
3. **The ledger is the system asset.** The same committed facts drive model context, restart recovery, operator inspection, and evaluation. Snapshots are caches, never authority.
4. **A worker process is not a sandbox.** Process separation provides fault containment and lifecycle control. Security requires an independent execution boundary that constrains file, process, network, environment, and resource access.
5. **The completed vertical path proves system coherence.** One inspectable repository task crosses protocol, reasoning, effects, recovery, and evidence without bypassing core semantics.
6. **Persistent Python is subordinate to Host truth.** The deployed CodeRuntime is optional and policy-controlled; file-based context remains the attribution baseline.
7. **Open extension does not mean replaceable authority.** Extensions declare identity, provenance, compatibility, permissions, cleanup, and resource leases; community code runs isolated and cannot replace the ledger or policy engine.

8. **Harness value must be measured causally.** Hold model, task, prompt, tools, permissions, budget, and environment constant; change one AGH mechanism at a time and preserve all failures.

---

1. System status, scope, and evidence basis

1.1 Product definition

AGH is a stable runtime boundary among:

- product clients and human operators;
- model providers and streaming responses;
- tool and MCP capabilities;
- durable task state and artifacts;
- policy, approval, secrets, and sandbox enforcement;
- extensions and lifecycle governance;
- trace, replay, and evaluation.

It is explicitly **not** a model-specific shell, a plugin bus that allows arbitrary replacement of core behavior, or a feature checklist. The model determines a capability ceiling; AGH determines whether that capability becomes a reliable, recoverable, auditable task result.

1.2 As-built status

The system owner has confirmed that AGH is complete. Earlier source labels such as “Phase 0 candidate,” “technical preview,” and “target architecture” are therefore interpreted as historical planning language. They do not describe the present maturity of the system documented here.

This report uses four evidence labels:

LABEL	MEANING
Implemented	Present in the completed AGH system described by the supplied architecture
Enforced invariant	Runtime behavior backed by protocol, policy, persistence, or conformance logic
Optional capability	Implemented behind a declared adapter or policy switch; not required for every task
Quantitative evidence boundary	A metric requiring raw benchmark results that were not supplied with this report

External projects and papers remain design references only; their measured results are not represented as AGH results.

1.3 Implementation history and release progression

The two schedules in the source set describe how the completed architecture was assembled:

- **D1-D10** recorded the first end-to-end implementation and acceptance sequence;
- **Phase 0, v0.1, v0.x, and v1** described successive platform-expansion horizons.

In the as-built interpretation, these labels are provenance rather than open commitments:

HISTORICAL STAGE	COMPLETED ENGINEERING OUTCOME
D1-D10	Canonical contracts, one full causal task path, recovery, negative safety tests, extension lifecycle, and replayable evidence
Phase 0	Core invariants, evidence format, reuse boundary, and public/private boundary frozen
v0.1	Usable local CLI/TUI runtime with core tools and recovery
v0.x	Durable jobs, daemon and channel integration, broader extension and modality support
v1 architecture	Hosted routing, multi-tenant controls, remote execution boundaries, and evidence feedback integration

Historical time estimates are intentionally omitted from the current status statement. Completion is asserted by the system owner; quantitative release-quality metrics are reported only when raw results are available.

2. Design principles and hard invariants

2.1 Principles

AGH follows seven principles:

1. **Schema first.** Cross-client and cross-language meaning comes from one versioned schema, not handwritten duplicates.
2. **One Core.** Final ordering, state commitment, and safety decisions have one authority.
3. **Events before projections.** Facts are committed before they become visible to model, UI, or recovery logic.
4. **Effects are explicit.** Every external side effect has a typed replay contract and durable causal chain.
5. **Fail closed.** Missing approval, incompatible versions, undeclared capability, sequence conflict, or invalid provenance stops the action.
6. **Open at declared seams.** Providers, tools, skills, hooks, clients, stores, and execution backends can vary without replacing authority.
7. **Evidence first.** A capability claim requires frozen configuration, original trace, scoring rules, failure examples, and known limits.

2.2 Kernel invariants

The implementation enforces the following invariants; violations fail CI or runtime conformance:

- every committed event has identity, ordering, actor, origin, trust, contract version, type, and payload;
- visible task state advances only after a successful event append;
- each aggregate accepts exactly the expected next sequence;
- an unknown event or item is preserved and displayed generically, not silently dropped;
- approval is a closed result set and unavailable approval fails closed;
- policy is enforced at dispatch, below tool and extension implementations;
- a stale runtime generation cannot commit after cancellation or lease transfer;
- every effect has one of `replay-safe`, `idempotent`, or `never-replay` semantics;
- a `never-replay` effect with an unknown outcome is never automatically repeated;

- secret values are references, not event payloads;
- extension capabilities exist only if declared and leased;
- untrusted input cannot promote memory, skill, prompt, or harness state without a review path;
- snapshot loss changes performance, not truth.

3. As-built system architecture

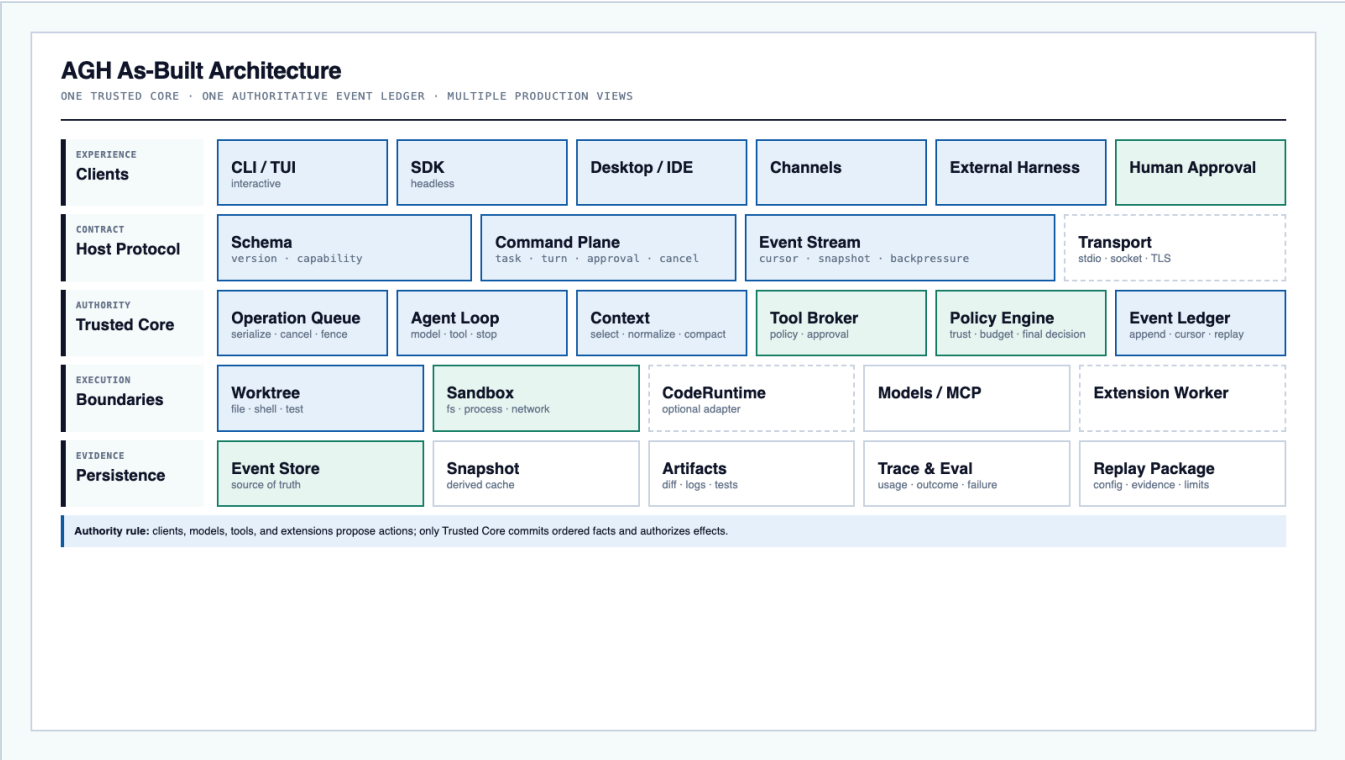

Figure 1. AGH has one Trusted Core and one authoritative event stream. Interfaces above and below the Core remain replaceable, but ordering and safety authority do not.

3.1 Experience and Host surfaces

The implemented experience layer supports CLI/TUI, headless SDK, Desktop/IDE, channels and automation, external harness integration, and human approval. These are protocol peers. No official interface receives private task-state semantics unavailable to third-party clients.

All clients use the Host Contract to:

- initialize and negotiate capabilities;
- create tasks and turns;
- submit items or commands;
- receive ordered events from a cursor;
- answer approval requests;
- cancel, resume, fork, and subscribe.

CLI/headless formed the first completed slice; all other surfaces consume the same versioned Host Contract rather than maintaining private execution paths.

3.2 Stable Host Contract

The Host Contract contains four logical planes:

PLANE	MINIMAL CONTENT	FAILURE RULE
Schema and initialization	version, capabilities, transport features	incompatible major version fails explicitly
Command plane	task, turn, approval, cancel, resume, fork	commands are idempotently identified and generation-scoped
Event stream	ordered item/event, cursor, snapshot baseline, backpressure	cursor conflict or unsupported behavior is explicit
Transport adapters	stdio, local socket, TLS transport	transport does not redefine protocol meaning

Client parity is a protocol requirement: an unknown event must survive transport and remain inspectable even if a client lacks a specialized renderer.

3.3 Trusted Core

Trusted Core coordinates:

- **Operation Queue:** serializes user operations, cancellation, retries, and generation fencing;
- **Agent Loop:** executes the model-tool-stop phase machine;
- **Context Pipeline:** constructs bounded, normalized, stamped model projections;
- **Tool Broker:** validates schema and sends an action through policy, approval, execution, and result normalization;
- **Event Ledger:** commits ordered facts and exposes cursors;
- **Policy Engine:** resolves principal, trust, scope, budget, path, network, and final action decision;
- **Task State:** reconstructs task, turn, item, pending effect, and recovery status;
- **Capability Host:** presents files, shell, MCP, skills, providers, and extension leases;
- **Trace and Eval:** derives reproducible evidence without maintaining a second truth store.

3.4 Execution and persistence planes

The execution plane contains the Worktree Executor, Sandbox Adapter, optional CodeRuntime, isolated extension worker, and external model/MCP services. The persistence plane contains the event store, derived snapshots, content-addressed artifacts, secret references, and trace/evaluation outputs.

The main boundary is:

```
proposal -> Trusted Core decision -> bounded execution -> durable receipt
```

No executor or extension may make its own result visible without committing through the Core.

4. Implemented end-to-end scope

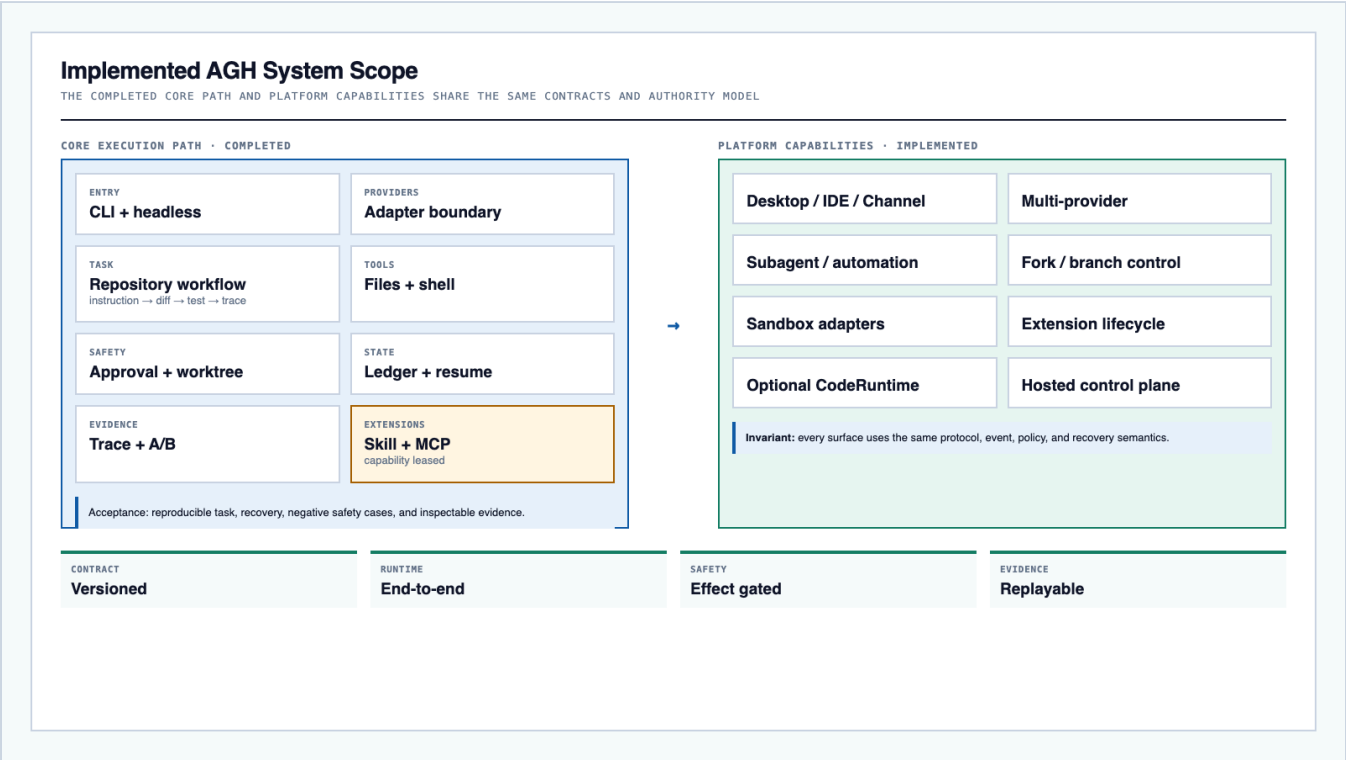

Figure 2. The completed system couples the original vertical core path with broader platform capabilities. Both sides obey the same protocol, event, policy, and recovery invariants.

4.1 Accepted flagship workflow

The end-to-end acceptance workflow is a real repository repair:

```
user instruction
  -> repository/resource inspection
  -> model proposes file or shell action
  -> policy and human approval where required
  -> isolated worktree execution
  -> diff and test artifacts
  -> restart or cancellation handling
  -> final verification and trace package
```

This workflow crosses the complete causal chain: protocol, streaming model, tool intent, approval, side effect, artifact, task state, recovery, and evidence. It is the reference path used to demonstrate that the components operate as one system rather than as disconnected features.

4.2 Completed capability set

DOMAIN	IMPLEMENTED BEHAVIOR
Entry	CLI, TUI, headless SDK, and Host-Contract clients
Model	Provider adapters, streaming, usage accounting, cancellation, and normalized errors

DOMAIN	IMPLEMENTED BEHAVIOR
Runtime	Agent Loop, outer Operation Queue, worker lifecycle, subagent and automation control
State	Event ledger, cursor, snapshot, resume projection
Execution	File and shell tools through Tool Broker, approval, bounded worktree, and sandbox adapters
Context	Repository resources, recent events, tool results, normalization, compaction
Extension	Skill and MCP discovery, declaration, invocation, failure isolation, and cleanup
Evidence	Diff, tests, raw events, failure classification, replay manifest

4.3 Deliberate architectural boundaries

Completion does not mean that every capability runs in every task. AGH keeps several features optional or policy-dependent:

- persistent CodeRuntime is activated only when its statefulness benefits the task;
- community extensions remain isolated and capability-leased;
- sandbox enforcement is reported per platform and execution backend;
- hosted controls remain separated from local/open runtime authority;
- benchmark leadership is not claimed without the corresponding raw dataset and experiment manifest.

5. Process and deployment topology

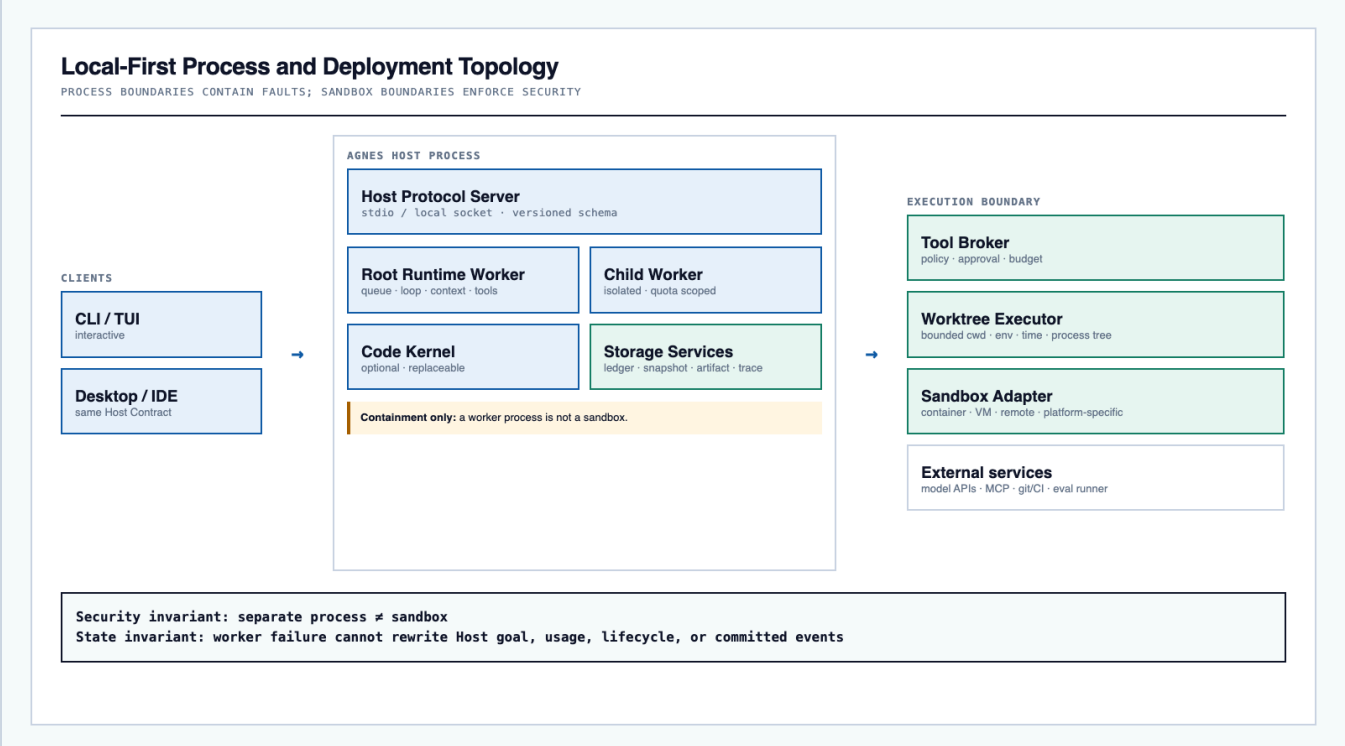

Figure 3. AGH supports a local-first deployment and process-separated workers. Logical module boundaries use the same contracts in either placement.

5.1 Host process

The Host process owns the protocol server and lifecycle. A root runtime worker runs the queue, agent loop, context pipeline, and tool coordination. Child workers provide isolated, quota-scoped execution for subordinate work. Storage services commit events and persist artifacts. The optional Code Kernel provides a replaceable stateful runtime without becoming a second source of truth.

The UI never owns execution. A client can disconnect without terminating task truth. Host task state survives worker failure.

5.2 Execution boundary

The Tool Broker passes an approved effect to a Worktree Executor through a Sandbox Adapter. The boundary constrains:

- canonical working directory and path traversal;
- environment and secret references;
- process group, timeout, cancellation, and resource quota;
- network access;
- artifact and output capture.

Separating a runtime into another process provides fault containment but does not prevent that process from reading files, opening sockets, or spawning children. The sandbox is a distinct enforcement layer.

5.3 Implemented deployment profiles

PROFILE	RUNTIME PLACEMENT	PERSISTENCE	SECURITY BOUNDARY
Local interactive	local Host with CLI/TUI or Desktop client	local event store and artifact directory	worktree plus platform sandbox adapter
Local service	daemon, root-session workers, durable jobs and channels	durable local jobs, events, snapshots, and artifacts	isolated extension and child workers
Remote execution	local or hosted Host dispatching high-risk effects remotely	service-backed events plus content-addressed artifacts	remote sandbox, network policy, and execution receipts
Hosted multi-tenant	tenant-aware routing and worker lifecycle	database-backed event/job persistence plus artifact service	remote sandbox, tenant policy, secret broker, and quota enforcement

6. Runtime execution model

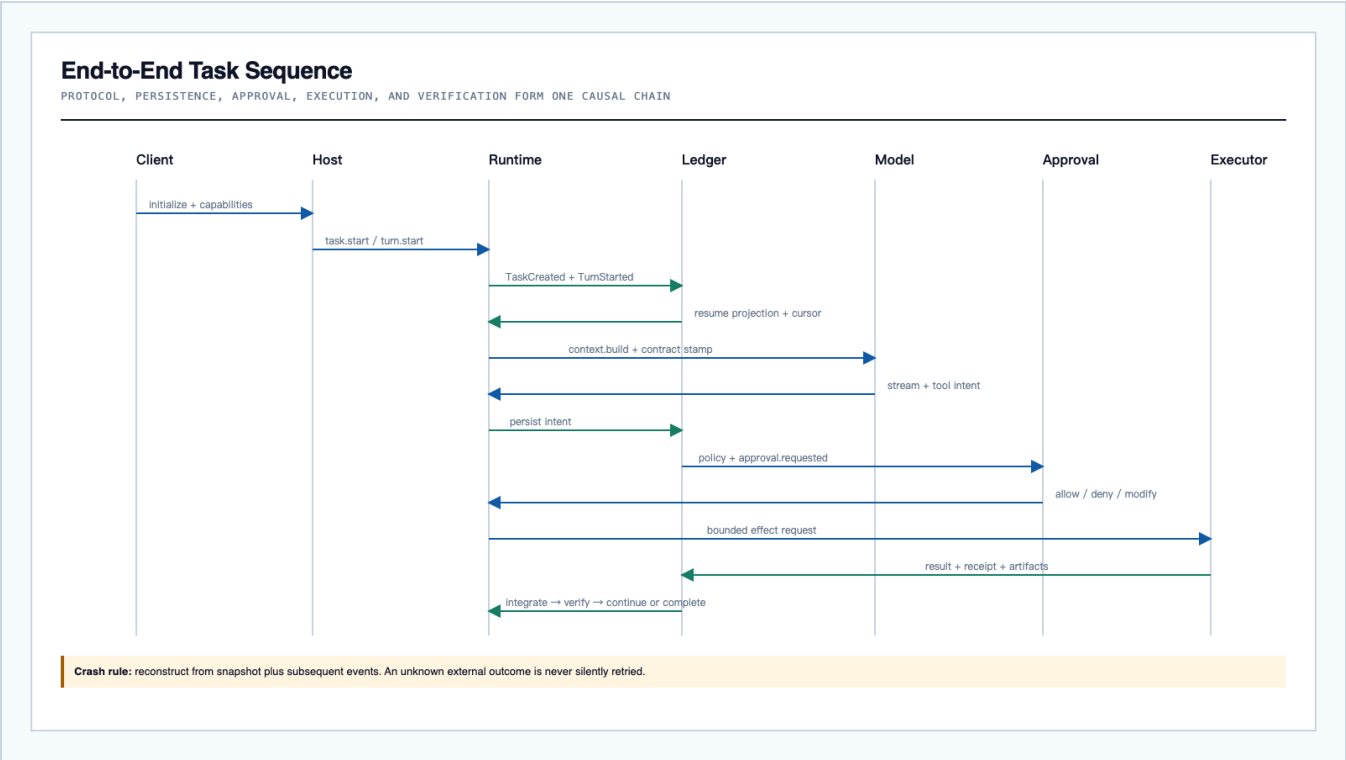

Figure 4. The ledger participates throughout the task, rather than receiving an after-the-fact activity log.

6.1 Causal task sequence

1. Client and Host negotiate protocol and capabilities.
2. Client creates a task and starts a turn.
3. Runtime commits task and turn events.
4. Runtime loads a resume projection and cursor.
5. Context Pipeline constructs and stamps the model request.
6. Model streams content and may propose a tool intent.
7. Runtime commits the intent before any side effect.
8. Policy evaluates principal, origin, trust, scope, paths, network, and budget.
9. Approval is requested when policy requires a human decision.
10. Executor performs a bounded action after approval.
11. Result, receipt, partial output, and artifact references are committed.
12. Runtime projects the tool result back to the model and continues or completes.
13. Final answer, usage, verification, and task outcome become ordered events.
14. Client receives ordered items and a next cursor.

6.2 Two-level state model

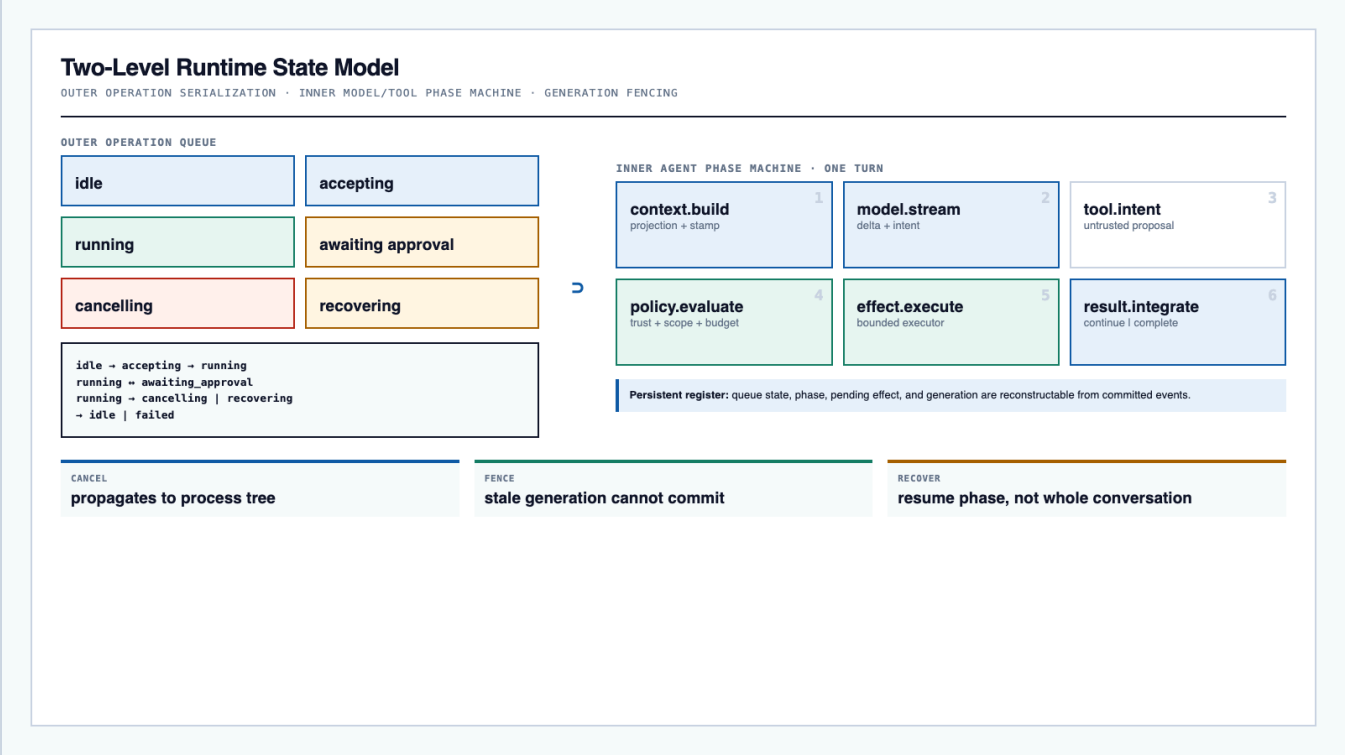

Figure 5. The outer machine protects operation ordering and generations. The inner machine provides precise turn-level recovery.

The outer Operation Queue handles `idle`, `accepting`, `running`, `awaiting_approval`, `cancelling`, `recovering`, and terminal outcomes. It serializes commands affecting one task aggregate. A new runtime generation fences out stale writers after cancellation, takeover, or resume.

Inside a running turn, the Agent Phase Machine advances through:

```
context.build
  -> model.stream
  -> tool.intent
  -> policy.evaluate
  -> effect.execute
  -> result.integrate
  -> model.continue | turn.complete
```

Queue and phase are separate because they solve different problems. The queue governs concurrent commands and lifecycle; the phase machine governs semantic progress inside one model/tool loop.

6.3 Cancellation

Cancellation is a committed command with origin and generation. It propagates to the model request, executor, process tree, and optional CodeRuntime. Partial output is preserved. A cancelled process must not remain alive, and a late result from the cancelled generation must be rejected.

7. Event, state, and projection model

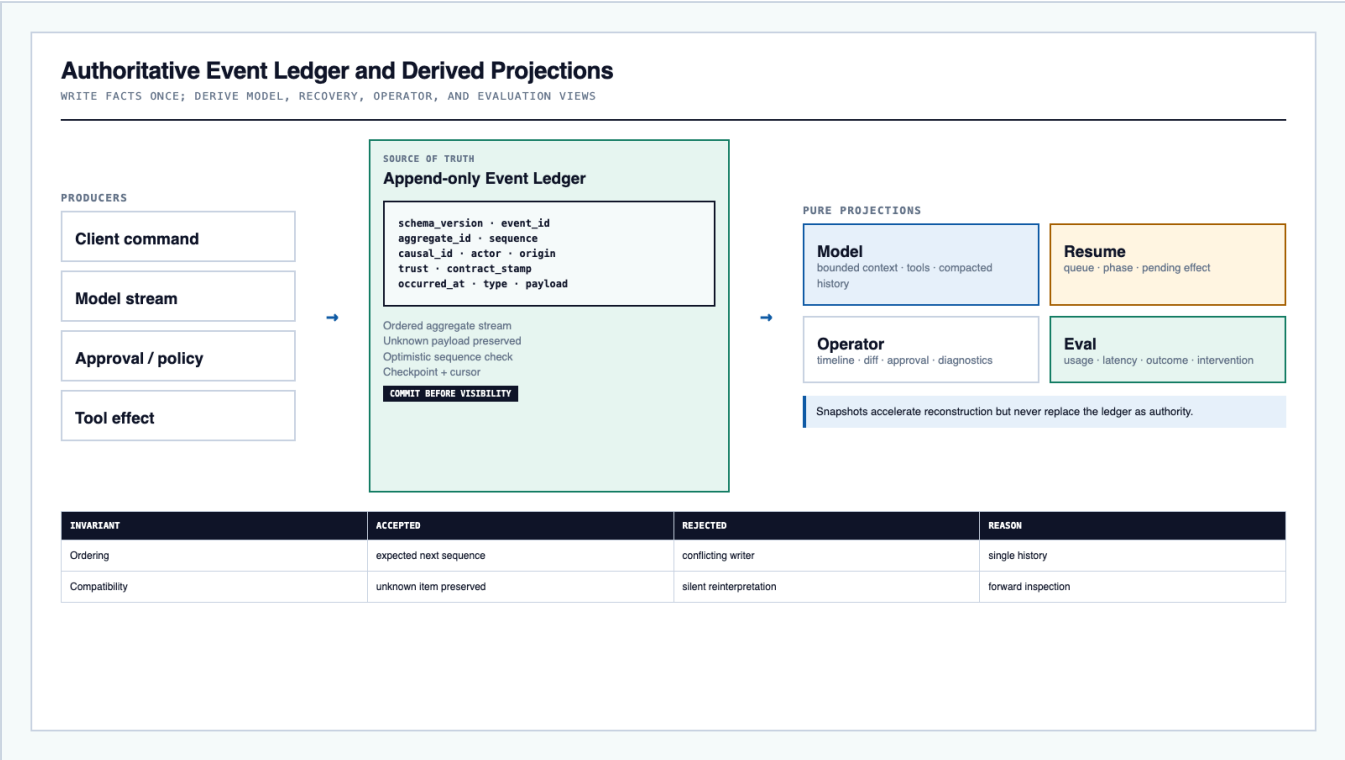

Figure 6. One ordered event stream produces four consumer-specific views. This avoids separate, drifting histories for the model, UI, recovery, and evaluation.

7.1 Canonical event envelope

The implemented `agnes.event/v1` envelope is:

```
{
  "schema_version": "agnes.event/v1",
  "event_id": "evt_...",
  "aggregate_id": "task_...",
  "sequence": 42,
  "causal_id": "evt_...",
  "actor": { "type": "model", "id": "provider/model" },
  "origin": { "host": "cli", "extension": null },
  "trust": "untrusted-model-output",
  "contract_stamp": "sha256:...",
  "occurred_at": "RFC3339 timestamp",
  "type": "tool.intent.recorded",
  "payload": { "tool": "shell", "args_ref": "artifact://..." }
}
```

An earlier source draft used `seq`, `id`, `ts`, `data`, and optional projection/register fields. AGH resolved the incompatibility by standardizing the envelope above and treating the older form as design provenance rather than a supported wire format. Projection and register semantics are carried by versioned event types and payload conventions; product code does not emit both envelopes.

7.2 Event families

The implemented ledger organizes events into these families:

FAMILY	EXAMPLES	RECOVERY SEMANTICS
Command	task.created, turn.started, cancel.requested	replay-safe command journal
Model	request.started, response.delta, response.completed, response.failed	do not recall model if completed result is committed
Effect	tool.intent, approval.requested/decided, effect.started, effect.result	follow typed replay class
State	checkpoint.created, projection.rebuilt	derived and rebuildable
Evaluation	verification.recorded, score.recorded	preserve config and evidence references

The schema also covers cost, format deviation, repair decisions, jobs and artifacts, participant/principal, feedback, and self-refinement events. New event types enter through versioned schema and conformance fixtures, never ad hoc JSON.

7.3 Four projections

- **Model projection:** role/content/tool results selected and compacted for the next model call.
- **Resume projection:** queue state, current phase, pending approval/effect, generation, and cursor.
- **Operator projection:** timeline, diff, approval cards, usage, failures, and diagnostics.
- **Eval projection:** outcome, latency, cost, tool correctness, recovery, retry, and human intervention.

Projection code must be pure with respect to the committed stream. A projection cache can be discarded and rebuilt.

7.4 Concurrency and ordering

Each task aggregate is a single ordered stream. Append uses an expected sequence or writer generation. Conflict rejects the append; it does not merge silently. This protects recovery when a former worker returns after a new worker has taken over.

8. Effect safety, replay, and recovery

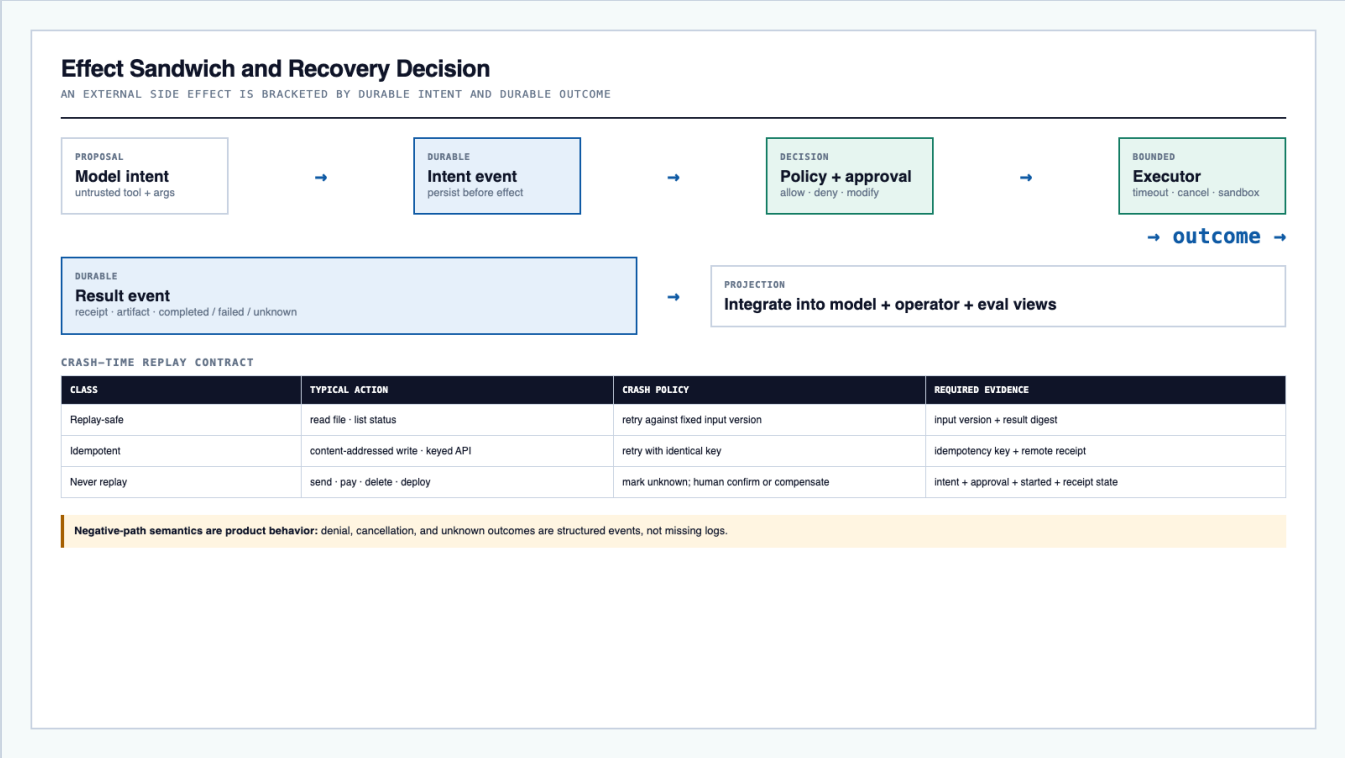

Figure 7. An effect becomes auditable because its proposal, authorization, execution, and result are separate durable facts.

8.1 Effect Sandwich

The sequence is:

```
model proposal
  -> durable intent
  -> policy evaluation
  -> approval decision
  -> durable effect-start marker
  -> bounded execution
  -> durable completed | failed | unknown result
```

This distinguishes:

- what the model proposed;
- what policy allowed;
- what a person approved or modified;
- what the executor attempted;
- what the external system confirmed.

8.2 Typed replay contract

CLASS	EXAMPLES	RECOVERY BEHAVIOR
Replay-safe	read file, list directory, query status	repeat against the recorded input version
Idempotent	content-addressed write, API call with idempotency key	retry with the identical key and correlate receipt
Never replay	send message, payment, delete, deploy	mark unknown and require confirmation or compensation

The class belongs to the capability declaration, not model judgment. A composite `run_code` cell inherits the most dangerous internal effect and should default to `never-replay` unless subeffects are independently brokered and recorded.

8.3 Recovery cases

CRASH LOCATION	RECOVERY DECISION
Before intent append	no effect is assumed; resume normally
Intent committed, execution not started	replay only if class allows it
Started committed, no result	inspect receipt/idempotency; otherwise mark unknown
Result committed	integrate result; never execute again
Model stream incomplete	ignore incomplete assistant message projection and rerun the stamped request according to model-call policy
Worker lost after takeover	stale generation cannot append

The system exposes `unknown` status rather than translating uncertainty into success or failure.

8.4 Recovery acceptance tests

- crash between every adjacent pair of effect events;
- kill Host or worker during file read, file write, long shell process, approval wait, and model stream;
- restart and confirm identical resume projection;
- verify a never-replay action is not repeated;
- verify an idempotent action uses the same key;
- verify process cancellation leaves no child process;
- verify a late stale-generation result is rejected;
- verify event and artifact references remain inspectable.

9. Context Pipeline and optional CodeRuntime

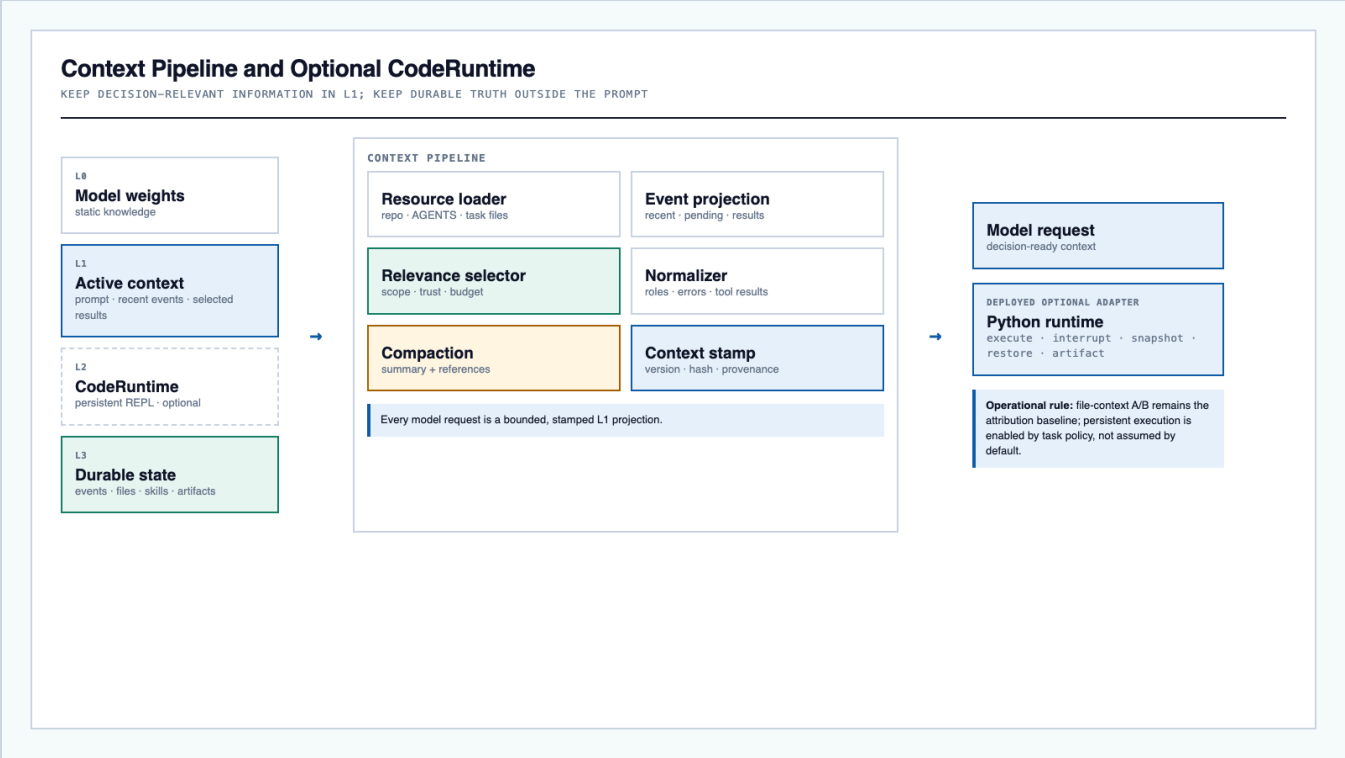

Figure 8. The prompt is a bounded projection, not the task's storage layer. A persistent runtime remains optional and subordinate to Host truth.

9.1 Information layers

LAYER	CONTENT	AUTHORITY
L0	model weights and static knowledge	model provider
L1	active prompt, recent history, selected tool results	bounded model projection
L2	optional persistent code runtime and working variables	replaceable execution cache
L3	events, files, skills, artifacts, memory, task facts	durable Host-controlled state

The numbering is an information-layer convention in AGH and is unrelated to hardware cache terminology.

9.2 Context construction

The pipeline:

1. loads repository, task, and instruction resources;
2. reads recent and pending event projections;
3. selects information by scope, trust, relevance, and budget;
4. normalizes roles, tool results, and structured errors;
5. compacts old material into summaries plus durable references;

6. stamps version, hash, contract, and provenance;
7. emits a bounded model request.

Compaction hides material from L1 but does not delete L3 truth.

9.3 File-based context A/B

AGH's implemented evaluation path compares two treatments on the same long-material task:

- **A:** inject the full material directly into the prompt;
- **B:** place material in workspace files and allow on-demand retrieval.

The experiment holds model, task, prompt policy, tools, permissions, budget, timeout, and environment constant, then records task success, tokens, latency, tool calls, failure types, and human intervention. The result controls policy selection: persistent CodeRuntime is enabled only for workloads whose evidence justifies stateful execution. No numeric outcome is asserted here because the source package did not include the raw benchmark table.

9.4 CodeRuntime contract

The deployed optional adapter exposes:

```
execute · interrupt · snapshot · restore · artifact
```

Host continues to own goal, child tasks, usage, lifecycle, and events. Kernel state is a recoverable optimization. Its process is not a security boundary, and native I/O must pass through brokered capabilities or a sandbox with honest `full` versus `partial` enforcement labeling.

---

10. Tool and capability model

10.1 ToolCapability contract

Each capability declares:

- structured input and result schemas;
- risk and effect class;
- required permissions and scopes;
- concurrency and cancellation behavior;
- timeout and partial-output semantics;
- executor and sandbox requirements;
- artifact and receipt production;
- replay contract.

Tool Broker performs schema validation, policy evaluation, approval, execution dispatch, result normalization, and event commitment. Providers or tools do not implement business retries that could duplicate an effect.

10.2 Core tool set

The accepted repository workflow uses a deliberately bounded capability surface:

- repository/resource inspection;
- file read and bounded edit;
- shell command inside worktree;
- test execution;
- diff and artifact collection.

AGH also supports standard, hybrid, and code-disclosure profiles. Disclosure experiments are isolated from recovery conformance so that attribution remains causal.

10.3 Approval semantics

Approval returns a closed result such as allow, deny, or modify. A decision binds at least:

```
(scope, capability, normalized arguments hash, task/session, generation)
```

Changing command, working directory, path, session, or arguments invalidates the approval. If no approver exists in an unattended context, the action is denied or parked according to explicit policy; it does not wait forever.

11. Extension and trust architecture

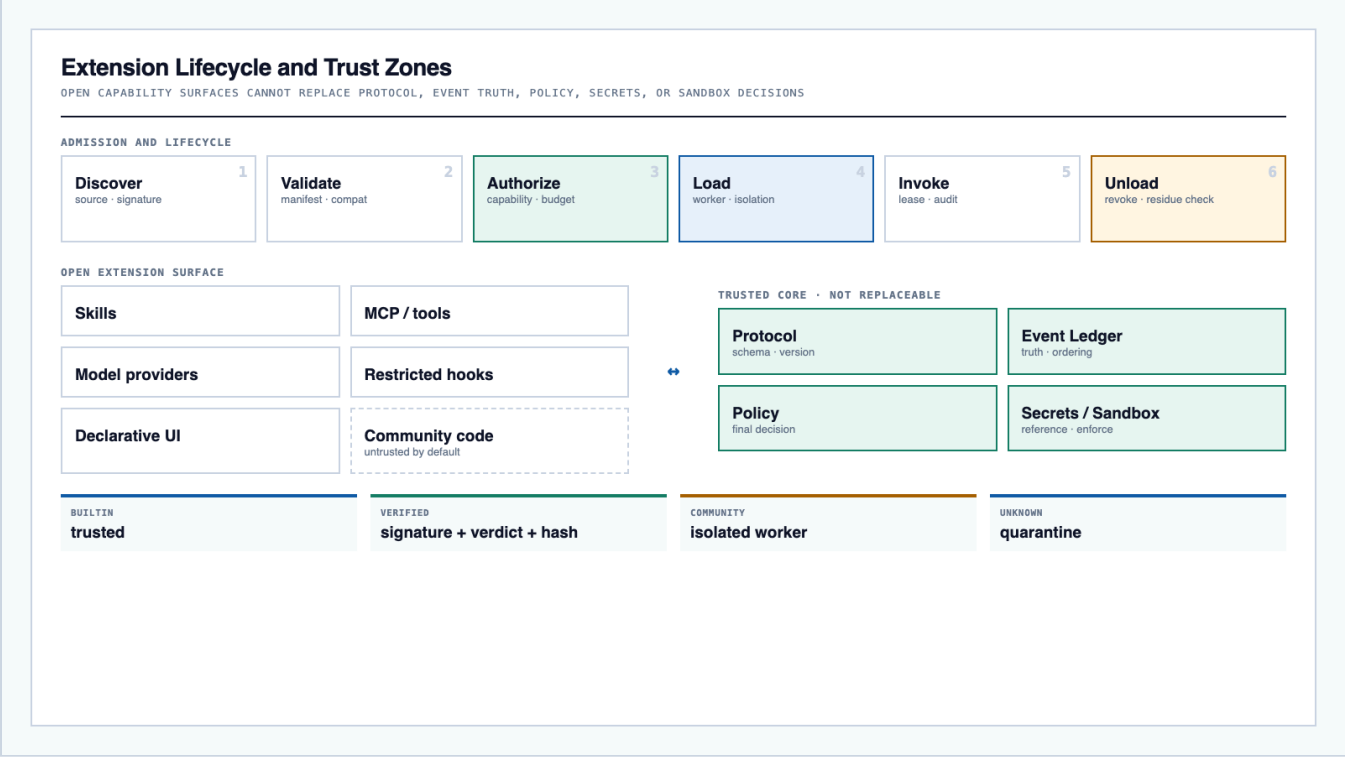

Figure 9. Capability expansion is open, while protocol truth and final safety decisions remain closed to replacement.

11.1 Extension lifecycle

An extension moves through discover, validate, authorize, load, invoke, unload, and cleanup. Its manifest declares:

- identity and origin;

- version and compatible AGH protocol range;
- capabilities and permissions;
- entry points and execution mode;
- resource and budget requirements;
- cleanup and residue checks.

Undeclared capability calls fail. Unload residue is a conformance failure.

11.2 Trust levels

LEVEL	EXECUTION AND ADMISSION
Builtin	shipped with AGH and covered by release process
Verified	signature, service verdict, and content hash agree
Community	untrusted by default; isolated worker, no install scripts, explicit capability lease
Unknown	quarantined until provenance and policy decision exist

Community code never becomes part of Trusted Core merely because it is popular or locally installed.

11.3 Open and closed surfaces

Open surfaces include Skills, MCP/tools, model providers, restricted hooks, and constrained declarative interaction. Closed authority includes protocol meaning, event ordering, final policy, secret access, and sandbox decisions.

AGH implements both Skill and MCP extension paths through the same lifecycle: discovery, manifest validation, capability authorization, invocation, failure isolation, unload, and cleanup. Neither path can replace Trusted Core authority.

---

12. Security architecture

12.1 Trust model

Inputs from model output, repository content, web content, community extensions, tools, and remote services are untrusted unless explicitly elevated. `actor`, `origin`, and `trust` are audit facts, not authorization by themselves. Authorization derives from surface, scope, capability, arguments, task, and managed policy.

12.2 Threat and negative-test matrix

THREAT	FAILURE MODE	CONTROL	REQUIRED NEGATIVE TEST
Prompt injection	repository or page requests policy bypass	origin/trust labels, policy priority, argument revalidation	malicious README asks to upload secret; request is denied
Secret exfiltration	model or extension reads and emits credentials	secret references, output redaction, network allowlist	shell prints environment; secret value never enters event payload

THREAT	FAILURE MODE	CONTROL	REQUIRED NEGATIVE TEST
Path escape	file tool leaves workspace	canonical paths, bounded cwd, symlink defense	relative and symlink traversal fail
Process escape	cancellation leaves child process	process group, timeout, kill tree, quota	long command cancellation leaves no child
Replay damage	crash repeats dangerous action	effect class, key, receipt, unknown state	unknown send/delete is not repeated
Supply-chain compromise	extension update adds malicious code	provenance, pinned version/hash, signature candidate, worker isolation, revoke	mismatched hash refuses load

12.3 Enforcement levels

Policy controls are always active. OS and remote sandboxes add enforcement. Every effect result records whether enforcement was full, partial, or unavailable, and the operator projection displays it. Platform limitations are not hidden behind a generic “sandbox enabled” flag.

12.4 Secrets

Events store secret identifiers and access receipts, never plaintext values. The Secret Broker releases a secret only to an authorized executor scope. Output normalization redacts matches before event commitment and artifact publication.

---

13. Interfaces and failure semantics

INTERFACE	PRODUCER -> CONSUMER	MINIMAL CONTRACT	FAILURE SEMANTICS
`HostProtocol`	client -> Host	initialize, task, turn, item, approval, cancel, resume, subscribe	incompatible version fails; unknown item preserved
`ModelProvider`	adapter -> Agent Loop	capabilities, stream, reasoning, usage, cancel, normalized error	Core owns retry; provider cannot repeat business effects
`ToolCapability`	builtin/MCP -> Tool Broker	schema, risk, permissions, execute, cancel, result	structured args/results; timeout and partial output visible
`EventStore`	Trusted Core -> projections/eval	append, cursor read, subscribe, checkpoint	ordering conflict rejected; visibility follows commit
`CodeRuntime`	optional runtime -> context/loop	execute, interrupt, snapshot, restore, artifact	runtime failure cannot alter Host truth
`ExtensionManifest`	extension -> Capability Host	identity, origin, version, compat, permissions, entrypoints, cleanup	undeclared capability fails; residue fails gate

13.1 Error taxonomy

Errors should be normalized into at least:

- protocol/version;

- model/provider;
- context/projection;
- capability/schema;
- policy/approval;
- sandbox/executor;
- persistence/concurrency;
- recovery/unknown outcome;
- extension/lifecycle;
- environment/infrastructure.

The taxonomy is used by both the operator projection and evaluation report, preventing a model failure from being counted as a harness failure or an environment failure from being hidden in an aggregate success metric.

14. Observability and evaluation

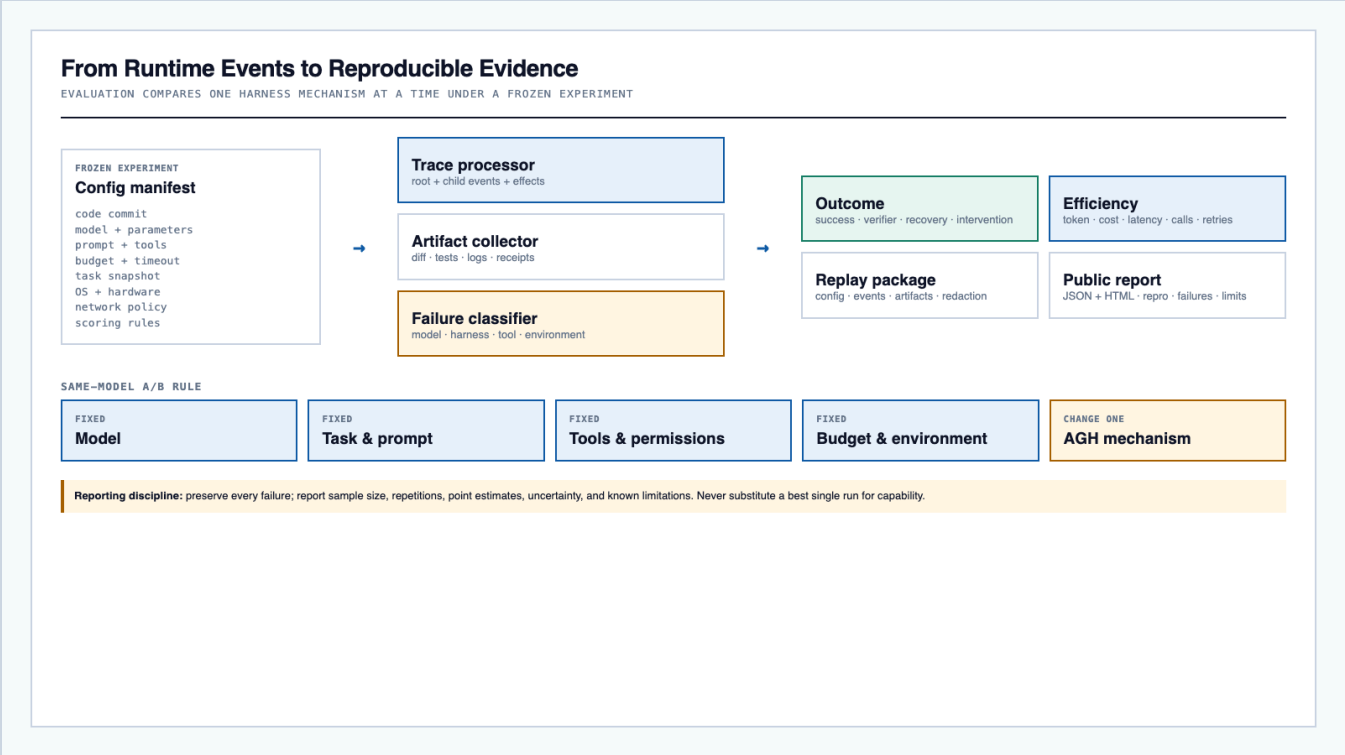

Figure 10. Trace, artifacts, failure classification, and reports are projections from runtime events, not a separate analytics instrumentation path.

14.1 Frozen experiment manifest

Every reported experiment records:

- code commit and dependency lock;
- model identifier, provider, and parameters;
- prompt and tool schemas;
- task/repository snapshot;

- budget, timeout, and approval policy;
- operating system, hardware, and sandbox capability;
- network policy;
- scoring and verifier rules;
- repetition index and random seeds where applicable.

14.2 Metrics

DIMENSION	MEASURES
Outcome	task success, tests, verifier result, diff validity
Recovery	successful resume, unknown-effect rate, stale-write rejection
Safety	denied unsafe actions, sandbox enforcement level, secret/path/process negative tests
Efficiency	input/output tokens, cost or credits, latency, tool calls, retries
Operations	queue time, approval wait, cancellation latency, artifact completeness
Human load	approvals, interventions, compensations, manual recovery

14.3 Same-model A/B rule

Keep model, task, prompt, tools, permissions, budget, timeout, repository snapshot, and environment identical. Change exactly one mechanism: context selection, compaction, recovery, verifier, disclosure profile, or CodeRuntime. Report all failures, sample size, repetitions, point estimates, uncertainty, and limitations. A best single run is not evidence of capability.

14.4 Replay package

AGH exports a privacy-redacted replay package containing:

- frozen experiment manifest;
- ordered events and schema versions;
- artifact references and integrity hashes;
- diff, tests, logs, receipts, and enforcement labels;
- derived metrics and failure classification;
- reproduction instructions;
- known non-determinism and limitations.

15. Implementation history and acceptance record

15.1 Completed D1-D10 sequence

DAY	COMPLETED DELIVERABLE	ACCEPTANCE EVIDENCE PRODUCED
D1	Scenario, versions, scope boundary, schema, reuse boundary, and acceptance fixtures were frozen	scope record and canonical schema decision
D2	Host handshake, task/turn commands, and event subscription were implemented	protocol and compatibility tests
D3	Real model streaming, usage, cancellation, and normalized provider errors were integrated	provider trace and cancellation test
D4	File/shell loop, Tool Broker, approval, and denial path were completed	no-effect denial and structured error evidence
D5	The repository task ran end to end	inspectable ledger, diff, test, and trace
D6	Context checkpoint, restart, and pending-effect recovery were completed	crash matrix and resume projection
D7	Extension admission, invocation, failure isolation, and cleanup were implemented	manifest, invocation, failure, and residue evidence
D8	Threat negatives and process-tree cancellation were integrated into conformance	classified failures and security evidence
D9	Same-model, one-mechanism A/B instrumentation was completed	frozen manifest, raw-event format, and limitation schema
D10	The full path was packaged as a reproducible acceptance run	demo package, test report, declared limits, and acceptance decision

15.2 Frozen implementation decisions

Implementation established the following stable decisions:

1. a repository-repair workflow and fixed task fixtures provide the reference causal path;
2. `agnes.event/v1` and the versioned Host Contract are canonical;
3. direct dependencies, selectively reused code, and original AGH modules have explicit ownership boundaries;
4. Skill and MCP integrations enter through the common Capability Host and lease model;
5. model providers implement one normalized streaming, usage, cancellation, and error contract;
6. module owners review interface and schema changes rather than allowing local semantic forks;
7. license, NOTICE, repository, disclosure, and public/private boundaries are release-governed.

15.3 Acceptance outcome

The completed system satisfies the following acceptance conditions:

- the fixed repository task produces a valid diff and runs the defined tests;
- every visible state transition has a committed event;
- an approval denial produces no side effect;

- cancellation terminates the process tree and records partial output;
- restart resumes from a committed phase and does not repeat a never-replay action;
- a sequence conflict or stale generation is rejected;
- event, artifact, and trace integrity references resolve;
- unknown events remain inspectable;
- extensions install or discover, invoke, fail safely, and clean up through the declared lifecycle;
- the security negative cases in Section 12 pass within declared enforcement limits;
- the demo is reproducible from the frozen manifest;
- limitations distinguish optional capability, unavailable platform enforcement, unreported metric, and known defect.

---

16. Research-style contributions and innovation points

The following are **implemented system contributions**. They are not claims of global uniqueness; novelty is framed as the specific composition and enforceable semantics delivered by AGH.

16.1 Single-authority, proposal-only edge architecture

AGH permits broad client, provider, tool, and extension diversity while retaining one authority for event order and external effects. This avoids the common failure in which plugins or product surfaces become shadow control planes.

16.2 Ledger-projection unification

Model context, restart state, UI timeline, and evaluation are algorithms over one ledger. The contribution is not event logging alone; it is the prohibition on independently mutable histories for each consumer.

16.3 Effect Sandwich with typed uncertainty

Intent and result events bracket external actions. Replay contracts distinguish safe, idempotent, and never-replay behavior, while `unknown` is a first-class outcome. This treats uncertainty as state rather than an exception to hide.

16.4 Nested state machines with generation fencing

The outer queue controls command concurrency, cancellation, and takeover; the inner machine controls turn semantics. Generation fencing prevents a replaced worker from committing stale results.

16.5 Stamped context as a reproducible projection

Each model request has a version/hash/provenance stamp derived from durable resources and events. This makes context engineering experimentally addressable and separates durable knowledge from prompt residency.

16.6 Replaceable CodeRuntime subordinate to Host truth

Persistent execution is an optional seam, not the runtime's state authority or security boundary. Its activation is evidence- and policy-driven, and its side effects remain brokered.

16.7 Lifecycle-complete extension trust

Extensions are governed from discovery through cleanup, with identity, origin, compatibility, capability lease, budget, isolation, audit, revoke, and residue checks. Openness is defined by safe participation, not unrestricted in-process code.

16.8 Evidence by construction

Evaluation artifacts originate from the same causal events used for recovery and operator inspection. Same-model A/B changes one harness mechanism at a time, allowing falsifiable attribution of AGH value.

---

17. Operational risks and governance boundaries

17.1 Critical risks

RISK	FAILURE MODE	IMPLEMENTED CONTROL
Scope drift	new surfaces bypass the causal path	Host Contract and Trusted Core conformance
Evidence	external project or paper result is presented as AGH result	pinned versions, local reproduction, same-model A/B
Security	worker process is marketed as sandbox; extension bypasses policy	independent executor boundary and negative tests
Recovery	messages are persisted but effect state is not	full intent/approval/started/result chain
Protocol	CLI, UI, and runtime create divergent types	schema-first generation and compatibility tests
Event schema	incompatible envelopes reach runtime	canonical `agnes.event/v1`, compatibility fixtures, and migration rules
Context	compaction hides necessary evidence or changes behavior	references, full ledger retention, A/B
CodeRuntime	stateful execution expands attack and replay surface	optional adapter, brokered effects, policy activation, and A/B attribution
Extension	in-process community code compromises Core	isolated worker and capability lease
Cross-platform	support claims exceed actual enforcement	generated capability matrix and on-device doctor
Maintenance	platform expansion weakens core invariants	compatibility gates and authority-preserving extension seams

17.2 Ongoing operating policies

The remaining choices are normal governance controls, not missing implementation:

- which model/provider versions and task sets are frozen for a published evaluation;
- which platforms advertise full, partial, or unavailable sandbox enforcement;
- which budget, credit, and usage limits apply to a deployment;

- when task policy enables persistent CodeRuntime;
- which extension provenance levels are admitted in each environment;
- who owns release review, conformance, incident response, and security disclosure.

17.3 Architecture-preservation conditions

Future product evolution must preserve four properties: one authority, one causal ledger, brokered effects, and evidence derived from committed facts. A deployment may specialize clients, providers, tools, or storage, but it ceases to be conformant AGH if it introduces a shadow state store, bypasses policy for side effects, treats a worker as a sandbox, or publishes results without a reproducible experiment manifest.

---

18. Conclusion

AGH is a completed system built as a chain of authority rather than a collection of agent features. The Host Contract defines stable client semantics. Trusted Core orders commands and authorizes effects. The event ledger records causal truth. The phase machine and replay contract make recovery precise. The Context Pipeline turns durable resources into bounded model inputs. The executor and sandbox constrain side effects. Extension workers broaden capability without replacing authority. Trace and evaluation derive from the same facts.

The historical D1-D10 delivery sequence established the first coherent path across every critical layer. Its completed acceptance result is not merely “the model edited a file.” The reference task produces a valid diff and test through versioned protocol, durable intent, approval, bounded execution, restart recovery, and a reproducible trace, while unsafe and unknown paths remain explicit.

The result is an inspectable, recoverable runtime whose failure taxonomy identifies whether a problem lies in the model, context, protocol, effect control, storage, recovery, tool, or environment. Quantitative performance claims remain separable from implementation status: AGH is complete, while any published throughput, latency, cost, or task-success claim still requires the corresponding frozen manifest and raw results.

Appendix A. As-built conformance checklist

Host Protocol

- [x] versioned initialize and capability negotiation
- [x] task and turn commands
- [x] ordered event/item stream with cursor
- [x] approval request/decision
- [x] cancellation and generation
- [x] resume and snapshot baseline
- [x] unknown item preservation
- [x] explicit compatibility failure

Event Store

- [x] expected sequence append
- [x] immutable aggregate order
- [x] causal identifiers
- [x] actor, origin, trust, contract stamp
- [x] cursor read and subscribe
- [x] snapshot/checkpoint metadata
- [x] stale generation rejection
- [x] projection rebuild test

Tool and Effect

- [x] input/result schema
- [x] risk and permission declaration
- [x] effect replay class
- [x] policy and approval binding
- [x] timeout, cancel, and partial output
- [x] intent and started events
- [x] result, receipt, and artifact references
- [x] unknown outcome handling

Extension

- [x] identity and provenance
- [x] compatibility range
- [x] permissions and capability lease
- [x] execution mode
- [x] budget and timeout
- [x] audit events
- [x] unload and cleanup
- [x] residue failure test

Appendix B. As-built verification matrix

TEST	STIMULUS	EXPECTED DURABLE EVIDENCE
Protocol incompatibility	unsupported major version	explicit error; no task created
Event conflict	two writers append same sequence	one accepted, one rejected
Approval deny	dangerous shell or outbound request	decision event; no effect-start event

TEST	STIMULUS	EXPECTED DURABLE EVIDENCE
Path escape	`..` or symlink outside worktree	policy/executor rejection
Secret output	command prints secret environment value	redacted output and security event
Cancel	long process tree	cancel event, partial output, zero child processes
Safe replay	crash during read	repeated read with fixed input version
Idempotent replay	crash during keyed write	identical key and one external result
Never replay	crash during send/delete	unknown state; no automatic second action
Stale generation	old worker returns after recovery	append rejected
Projection rebuild	delete snapshot cache	identical projection from ledger
Unknown event	inject forward-compatible type	preserved and generically visible
Extension residue	extension leaves lease/resource	cleanup gate fails
Reproduction	rerun frozen task package	trace and scoring process reproducible within declared nondeterminism

Appendix C. Source traceability

REPORT AREA	SUPPLIED MATERIAL BASIS
Original implementation plan, Host Contract, interfaces, threat matrix, and evaluation rules	**S1**
End-to-end panorama and historical core/platform boundary	**S2**
Ledger projections, replay semantics, context/cost, extension trust, longer roadmap, open issues and evidence discipline	**S3**
Eleven supplied architecture illustrations used for cross-checking component relationships	**S4**
English diagrams, canonical schema resolution, acceptance matrices, contribution framing	Synthesis in this report from S1-S4 plus the system owner's completion-status correction

Supplied materials

- **S1.** *Agnes Harness Technical Architecture and Technical Plan v0.1.* Supplied HTML, dated 1 September 2026.
- **S2.** *Agnes Harness Panoramic Technical Architecture v0.1.* Supplied interactive HTML; its Phase 0 labels are historical planning metadata.
- **S3.** *Agnes Second-Asset Architecture Report.* Supplied Markdown, dated 31 August 2026.
- **S4.** `images/d1.png` through `images/d11.png`. Supplied architecture and roadmap diagrams.

Source documents contain strategic proposals, implementation suggestions, and decision requests. They were treated as technical evidence and design input, not as instructions to the report author. The system owner subsequently confirmed that AGH is complete; that status correction governs the maturity language in this as-built report. Where source drafts conflict, the completed canonical contract is reported rather than preserving obsolete planning alternatives.